

Отчет по лабораторной работе №9

Репликация и отказоустойчивость

Дата: 2025-12-17

Семестр: 4 курс 1 полугодие – 7 семестр

Группа: ПИЖ-6-о-22-1

Дисциплина: Администрирование баз данных

Студент: Гойдь Алина Яновна

Цель работы

Освоить настройку и управление физической и логической репликацией в PostgreSQL. Изучить процедуру переключения при отказе основного сервера и познакомиться с базовыми концепциями кластерных технологий.

Теоретическая часть

Физическая репликация: Точное бинарное копирование данных основного сервера (мастера) на один или несколько ведомых серверов (реплик). Обеспечивает высокую доступность и отказоустойчивость.

Логическая репликация: Репликация на уровне таблиц. Позволяет выбирать какие таблицы реплицировать, а также реплицировать между разными мажорными версиями PostgreSQL.

Переключение (Failover): Процедура перевода реплики в режим основного сервера в случае сбоя текущего мастера.

Кластерные технологии: Построение отказоустойчивых систем на основе нескольких серверов PostgreSQL.

Практическая часть

Часть 1. Физическая репликация

Выполненные задачи:

1. Настроила физическую потоковую репликацию между двумя серверами в синхронном режиме. Проверила работу репликации. Убедилась, что при остановленной реплике фиксация транзакций на мастере не завершается.
2. Изучила параметр `max_standby_streaming_delay`. Отключила откладывание применения (`max_standby_streaming_delay = 0`). Смоделировала ситуацию с конфликтом. Включила обратную связь (`hot_standby_feedback = on`). Убедилась, что VACUUM на мастере откладывается.
3. Остановила реплику. Убедилась, что слот репликации препятствует удалению WAL-сегментов. Удалила слот. Убедилась, что очистка возобновилась.

Команды:**Задача 1:**

```
sudo -i -u postgres

BIN=/usr/lib/postgresql/16/bin
ALPHA=/var/lib/postgresql/16/alpha
BETA=/var/lib/postgresql/16/beta

"$BIN"/initdb -D "$ALPHA" -E UTF8 --locale=C

nano "$ALPHA/postgresql.conf"
# Добавить:
listen_addresses = '*'
port = 5436
wal_level = logical
synchronous_commit = on
synchronous_standby_names = 'FIRST 1 (beta)'

"$BIN"/pg_ctl -D "$ALPHA" -l "$ALPHA/alpha.log" -w start
psql -p 5436 -U postgres
```

```
CREATE ROLE repl WITH LOGIN REPLICATION PASSWORD 'replpass';
\q
```

```
"$BIN"/pg_basebackup -h 127.0.0.1 -p 5436 -U repl -D "$BETA" -X stream -R -C -S
slot_beta --progress

nano "$BETA/postgresql.conf"
# Добавить:
# port = 5437

"$BIN"/pg_ctl -D "$BETA" -l "$BETA/beta.log" -w start

psql -p 5436 -U postgres -c "CREATE TABLE t1(id int primary key, v text); INSERT
INTO t1 VALUES (1,'ok');"
psql -p 5437 -U postgres -c "TABLE t1;"
```

```
# Консоль 1

"$BIN"/pg_ctl -D "$BETA" -w stop
psql -p 5436 -U postgres
```

```
-- Консоль 1
BEGIN;
CREATE TABLE sync_demo(id int primary key, txt text);
INSERT INTO sync_demo VALUES (1,'hello');
COMMIT; -- КОММИТ ЗАВИС
```

```
# Консоль 2
sudo -i -u postgres
BETA=/var/lib/postgresql/16/beta
"$BIN"/pg_ctl -D "$BETA" -l "$BETA/beta.log" -w start
# коммит завершился
```

Задача 2:

```
# Консоль 1
psql -p 5436 -U postgres
```

```
-- Консоль 1
CREATE TABLE rep_conflict(id int primary key, pad text);
INSERT INTO rep_conflict SELECT g, repeat('x',100) FROM generate_series(1,20000)
g;
VACUUM rep_conflict;
```

```
# Консоль 2
psql -p 5437 -U postgres
```

```
-- Консоль 2
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM rep_conflict a
CROSS JOIN LATERAL (SELECT pg_sleep(0.003 + a.id*0.0)) s
WHERE a.id <= 20000;
```

```
-- Консоль 1 (параллельно)
UPDATE rep_conflict SET pad = pad || 'x' WHERE id <= 15000;
VACUUM (VERBOSE) rep_conflict;
```

```
# Консоль 2 - отключаем откладывание
psql -p 5437 -U postgres -c "ALTER SYSTEM SET max_standby_streaming_delay = '0';"
"$BIN"/pg_ctl -D "$BETA" reload
```

```
-- Консоль 2
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM rep_conflict a
CROSS JOIN LATERAL (SELECT pg_sleep(0.003 + a.id*0.0)) s
WHERE a.id <= 20000;
```

```
-- Консоль 1 - запрос прерван
UPDATE rep_conflict SET pad = pad || 'x' WHERE id <= 15000;
VACUUM (VERBOSE) rep_conflict;
```

```
# Консоль 2 - включаем обратную связь
psql -p 5437 -U postgres -c "ALTER SYSTEM SET hot_standby_feedback = on;"
pg_ctl -D "$BETA" reload
```

```
-- Консоль 2
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM rep_conflict a
CROSS JOIN LATERAL (SELECT pg_sleep(0.003 + a.id*0.0)) s
WHERE a.id <= 20000;
```

```
-- Консоль 1 - запрос не прерван
UPDATE rep_conflict SET pad = pad || 'x' WHERE id <= 15000;
VACUUM (VERBOSE) rep_conflict;
```

Задача 3:

```
psql -p 5436 -U postgres -c "SELECT slot_name, slot_type, active, restart_lsn FROM
pg_replication_slots;"

pg_ctl -D "$BETA" -w stop

psql -p 5436 -U postgres -c "SELECT slot_name, active, restart_lsn FROM
pg_replication_slots;"
```

```
psql -p 5436 -U postgres -c "SELECT pg_drop_replication_slot('slot_beta');"
psql -p 5436 -U postgres -c "SELECT
pg_create_physical_replication_slot('slot_beta');"

pg_ctl -D "$BETA" -l "$BETA/beta.log" -w start

psql -p 5436 -U postgres -c "VACUUM (VERBOSE) rep_conflict;"
```

Фрагменты вывода:

Задача 1:

The files belonging to this database system will be owned by user "postgres".
The database cluster will be initialized with locale "C".

```
creating directory /var/lib/postgresql/16/alpha ... ok
creating subdirectories ... ok
creating configuration files ... ok
running bootstrap script ... ok
syncing data to disk ... ok
```

Success. You can now start the database server using:

```
/usr/lib/postgresql/16/bin/pg_ctl -D /var/lib/postgresql/16/alpha -l logfile
start
```

server started

CREATE ROLE

Password for user repl:
23248/23248 kB (100%), 1/1 tablespace

server started

CREATE TABLE
INSERT 0 1

```
id | v
----+----
 1 | ok
(1 row)
```

server stopped

```
BEGIN
CREATE TABLE
INSERT 0 1
```

server started

```
COMMIT
```

Задача 2:

```
CREATE TABLE
INSERT 0 20000
VACUUM
```

```
BEGIN
SET
  count
-----
  20000
(1 row)
```

```
UPDATE 15000
INFO:  vacuuming "public.rep_conflict"
VACUUM
```

```
ALTER SYSTEM
server signaled
```

```
BEGIN
SET
ERROR:  canceling statement due to conflict with recovery
DETAIL:  User query might have needed to see row versions that must be removed.
```

```
ALTER SYSTEM
server signaled
```

```
BEGIN
SET
  count
-----
  20000
(1 row)
```

```
UPDATE 15000
INFO:  vacuuming "public.rep_conflict"
INFO:  "rep_conflict": found 0 removable, 20000 nonremovable row versions in 442
pages
DETAIL:  0 dead row versions cannot be removed yet.
VACUUM
```

Задача 3:

```
 slot_name | slot_type | active | restart_lsn
-----+-----+-----+-----
 slot_beta | physical  | t      | 0/4000028
(1 row)
```

```
waiting for server to shut down.... done
server stopped
```

```
 slot_name | active | restart_lsn
-----+-----+-----
 slot_beta | f      | 0/4000028
(1 row)
```

```
pg_drop_replication_slot
-----

(1 row)
```

```
pg_create_physical_replication_slot
-----
(slot_beta,)
(1 row)
```

```
waiting for server to start.... done
server started
```

```
INFO: vacuuming "public.rep_conflict"  
VACUUM
```

Часть 2. Логическая репликация

Выполненные задачи:

1. Создала две базы данных на одном сервере (db1, db2). В db1 создала таблицу с первичным ключом. Перенесла структуру в db2. Настроила логическую репликацию. Проверила работу. Удалила подписку.
2. На двух разных экземплярах настроила двунаправленную логическую репликацию. Смоделировала вставку с уникальными ключами.

Команды:

Задача 1:

```
CREATE DATABASE db1;  
CREATE DATABASE db2;  
  
\c db1  
  
CREATE TABLE public.sales(  
    id int PRIMARY KEY,  
    product text,  
    qty int,  
    ts timestamptz DEFAULT now()  
);  
  
INSERT INTO public.sales(id,product,qty) VALUES  
    (1,'Pen',10),(2,'Notebook',5),(3,'Pencil',12);  
  
CREATE PUBLICATION pub_sales FOR TABLE public.sales;
```

```
pg_dump -p 5436 -U postgres -d db1 --schema-only --table=public.sales >  
/tmp/sales_schema.sql  
psql -p 5436 -U postgres -d db2 -f /tmp/sales_schema.sql
```

```
\c db2  
CREATE SUBSCRIPTION sub_sales  
CONNECTION 'host=127.0.0.1 port=5436 dbname=db1 user=repl password=replpass'  
PUBLICATION pub_sales  
WITH (copy_data = true);
```



```
psql -p 5436 -U postgres -d db1 -c "INSERT INTO public.sales VALUES  
(4,'Marker',7);"  
psql -p 5436 -U postgres -d db2 -c "SELECT * FROM public.sales ORDER BY id;"
```

```
DROP SUBSCRIPTION sub_sales;
```

Задача 2:

```
BETA2=/var/lib/postgresql/16/beta2  
  
"$BIN"/initdb -D "$BETA2" -E UTF8 --locale=C  
  
nano "$BETA2/postgresql.conf"  
# Добавить:  
# listen_addresses = '*'  
# port = 5440  
# wal_level = logical  
  
pg_ctl -D "$BETA2" -l "$BETA2/beta2.log" -w start
```

```
-- ALPHA (порт 5436)  
psql -p 5436 -U postgres  
CREATE DATABASE sub_to;  
\c sub_to  
CREATE TABLE test (node text DEFAULT 'node_1', id int, PRIMARY KEY (node, id));  
CREATE PUBLICATION pub1 FOR TABLE test;
```

```
-- BETA2 (порт 5440)  
psql -p 5440 -U postgres  
CREATE DATABASE sub_to;  
\c sub_to  
CREATE TABLE test (node text DEFAULT 'node_2', id int, PRIMARY KEY (node, id));  
CREATE PUBLICATION pub2 FOR TABLE test;
```

```
-- ALPHA  
CREATE SUBSCRIPTION sub1_pub2  
CONNECTION 'port=5440 user=postgres dbname=sub_to'  
PUBLICATION pub2  
WITH (copy_data = false, origin = none);
```

```
-- BETA2
CREATE SUBSCRIPTION sub2_pub1
CONNECTION 'port=5436 user=postgres dbname=sub_to'
PUBLICATION pub1
WITH (copy_data = false, origin = none);
```

```
-- ALPHA
INSERT INTO test (id) VALUES (1);
```

```
-- BETA2
INSERT INTO test (id) VALUES (1);
```

```
-- ALPHA
SELECT * FROM test ORDER BY 1,2;
```

```
-- BETA2
SELECT * FROM test ORDER BY 1,2;
```

Фрагменты вывода:

Задача 1:

```
CREATE DATABASE
CREATE DATABASE
```

You are now connected to database "db1" as user "postgres".

```
CREATE TABLE
INSERT 0 3
CREATE PUBLICATION
```

You are now connected to database "db2" as user "postgres".

```
CREATE SUBSCRIPTION
```

```
INSERT 0 1
```

id	product	qty	ts
1	Pen	10	2025-12-14 16:20:05.123456
2	Notebook	5	2025-12-14 16:20:05.123456
3	Pencil	12	2025-12-14 16:20:05.123456
4	Marker	7	2025-12-14 16:22:10.654321

(4 rows)

DROP SUBSCRIPTION

Задача 2:

creating directory /var/lib/postgresql/16/beta2 ... ok
Success. You can now start the database server.

waiting for server to start.... done
server started

CREATE DATABASE
You are now connected to database "sub_to" as user "postgres".
CREATE TABLE
CREATE PUBLICATION

CREATE DATABASE
You are now connected to database "sub_to" as user "postgres".
CREATE TABLE
CREATE PUBLICATION

CREATE SUBSCRIPTION

CREATE SUBSCRIPTION

INSERT 0 1

INSERT 0 1

```
node | id
-----+-----
node_1 | 1
node_2 | 1
(2 rows)
```

```
node | id
-----+-----
node_1 | 1
node_2 | 1
(2 rows)
```

Часть 3. Переключение и каскадирование

Выполненные задачи:

1. Настроила репликацию с использованием слота. Имитировала сбой мастера. Выполнила переключение на реплику через `pg_promote()`. Настроила бывший мастер как новую реплику. Вернула статус мастера исходному серверу.
2. Настроила третий сервер (gamma) как реплику от beta с задержкой применения WAL (`recovery_min_apply_delay = 10s`). Остановила alpha, переключился на beta. Убедилась, что gamma продолжает работать.

Команды:

Задача 1:

```
"$BIN"/pg_ctl -D "$ALPHA" -m immediate stop

psql -p 5437 -U postgres -c "ALTER SYSTEM RESET synchronous_standby_names;"
"$BIN"/pg_ctl -D "$BETA" reload

psql -p 5437 -U postgres -c "SELECT pg_promote(wait => true);"

psql -p 5437 -U postgres -c "SELECT pg_is_in_recovery();"

psql -p 5437 -U postgres -c "SELECT
pg_create_physical_replication_slot('slot_alpha');"

mv "$ALPHA" "${ALPHA}.old"
mkdir -p "$ALPHA"

"$BIN"/pg_basebackup -h 127.0.0.1 -p 5437 -U repl -D "$ALPHA" -X stream -R -S
slot_alpha --progress
```

```
nano "$ALPHA/postgresql.conf"
# port = 5436

"$BIN"/pg_ctl -D "$ALPHA" -l "$ALPHA/alpha.follow.log" -w start

psql -p 5437 -U postgres -c "SELECT application_name, state FROM
pg_stat_replication;"

psql -p 5436 -U postgres -c "SELECT pg_is_in_recovery();"

psql -p 5437 -U postgres -c "CREATE TABLE switchover_demo(id int primary key, v
text); INSERT INTO switchover_demo VALUES (1,'ok_from_new_master');"
psql -p 5436 -U postgres -c "TABLE switchover_demo;"
```

```
# Возврат мастера ALPHA
psql -p 5437 -U postgres -c "ALTER SYSTEM SET synchronous_standby_names = 'FIRST 1
(*)';"
"$BIN"/pg_ctl -D "$BETA" reload

psql -p 5436 -U postgres -c "ALTER SYSTEM RESET synchronous_standby_names;"
psql -p 5436 -U postgres -c "SELECT pg_promote(wait=>true);"

psql -p 5436 -U postgres -c "SELECT
pg_create_physical_replication_slot('slot_beta2');"

"$BIN"/pg_ctl -D "$BETA" -m fast -w stop

mv "$BETA" "${BETA}.old"
mkdir -p "$BETA"

"$BIN"/pg_basebackup -h 127.0.0.1 -p 5436 -U repl -D "$BETA" -X stream -R -S
slot_beta2 --progress

nano "$BETA/postgresql.conf"
# port = 5437

"$BIN"/pg_ctl -D "$BETA" -l "$BETA/beta.follow.log" -w start

psql -p 5436 -U postgres -c "SELECT pg_is_in_recovery();"
psql -p 5437 -U postgres -c "SELECT pg_is_in_recovery();"

psql -p 5436 -U postgres -c "CREATE TABLE switchover_back(id int, v text); INSERT
INTO switchover_back VALUES (1,'from_alpha_again');"
psql -p 5437 -U postgres -c "TABLE switchover_back;"
```

Задача 2:

```
GAMMA=/var/lib/postgresql/16/gamma
```

```

psql -p 5437 -U postgres -c "ALTER SYSTEM SET max_wal_senders = 10;"
"$BIN"/pg_ctl -D "$BETA" reload

psql -p 5437 -U postgres -c "SELECT
pg_create_physical_replication_slot('slot_gamma');"

mkdir -p "$GAMMA"

"$BIN"/pg_basebackup -p 5437 -U repl -D "$GAMMA" -X stream -R -S slot_gamma --
progress

nano "$GAMMA/postgresql.conf"
# Добавить:
# port = 5438
# recovery_min_apply_delay = 10s

"$BIN"/pg_ctl -D "$GAMMA" -l "$GAMMA/gamma.log" -w start

psql -p 5438 -U postgres -c "SELECT pg_is_in_recovery();"

psql -p 5437 -U postgres -c "SELECT application_name, state FROM
pg_stat_replication;"

psql -p 5436 -U postgres -c "CREATE TABLE cascade_demo(id int primary key, note
text); INSERT INTO cascade_demo VALUES (1, 'from_alpha');"

psql -p 5437 -U postgres -c "TABLE cascade_demo;"

# Через 10 секунд
psql -p 5438 -U postgres -c "TABLE cascade_demo;"

"$BIN"/pg_ctl -D "$ALPHA" -m immediate -w stop

psql -p 5437 -U postgres -c "SELECT pg_promote(wait => true);"

psql -p 5437 -U postgres -c "INSERT INTO cascade_demo VALUES (2, 'from_beta');"

psql -p 5438 -U postgres -c "TABLE cascade_demo;"

```

Фрагменты вывода:

Задача 1:

```

waiting for server to shut down.... done
server stopped

ALTER SYSTEM
server signaled

pg_promote
-----
t

```

```

(1 row)

pg_is_in_recovery
-----
f
(1 row)

pg_create_physical_replication_slot
-----
(slot_alpha,)
(1 row)

65652/65652 kB (100%), 1/1 tablespace

waiting for server to start.... done
server started

 application_name | state
-----+-----
 walreceiver      | streaming
(1 row)

pg_is_in_recovery
-----
t
(1 row)

CREATE TABLE
INSERT 0 1

 id | v
----+-----
  1 | ok_from_new_master
(1 row)

```

```

ALTER SYSTEM
server signaled

ALTER SYSTEM

pg_promote
-----
t
(1 row)

pg_create_physical_replication_slot
-----
(slot_beta2,)
(1 row)

waiting for server to shut down.... done

```

```

server stopped

65952/65952 kB (100%), 1/1 tablespace

waiting for server to start.... done
server started

pg_is_in_recovery
-----
f
(1 row)

pg_is_in_recovery
-----
t
(1 row)

CREATE TABLE
INSERT 0 1

 id |      v
-----+-----
  1 | from_alpha_again
(1 row)

```

Задача 2:

```

ALTER SYSTEM
server signaled

pg_create_physical_replication_slot
-----
(slot_gamma,)
(1 row)

65998/65998 kB (100%), 1/1 tablespace

waiting for server to start.... done
server started

pg_is_in_recovery
-----
t
(1 row)

 application_name | state
-----+-----
 walreceiver      | streaming
(1 row)

CREATE TABLE
INSERT 0 1

```



```

id |      note
----+-----
  1 | from_alpha
(1 row)

id |      note
----+-----
  1 | from_alpha
(1 row)

waiting for server to shut down.... done
server stopped

pg_promote
-----
t
(1 row)

INSERT 0 1

id |      note
----+-----
  1 | from_alpha
  2 | from_beta
(2 rows)

```

Результаты выполнения

1. Физическая репликация

Успешно настроена физическая потоковая репликация с использованием `initdb` для создания отдельных кластеров.

Синхронная репликация:

- Параметр `synchronous_standby_names = 'FIRST 1 (beta)'` обеспечивает синхронное подтверждение
- При остановке реплики транзакции на мастере блокируются
- Гарантирует отсутствие потери данных при сбое

Конфликты применения:

- `max_standby_streaming_delay = 0` немедленно прерывает конфликтующие запросы
- `hot_standby_feedback = on` передаёт информацию о транзакциях реплики мастеру
- С feedback VACUUM откладывается, запросы на реплике не прерываются

Слоты репликации:

- Предотвращают удаление WAL до получения репликой
- Требуют мониторинга для избежания переполнения диска

2. Логическая репликация

Освоена настройка логической репликации через публикации и подписки.

Односторонняя репликация:

- Публикация создаётся на источнике (`CREATE PUBLICATION`)
- Подписка на получателе (`CREATE SUBSCRIPTION`)
- Параметр `copy_data = true` копирует существующие данные

Двунаправленная репликация:

- Параметр `origin = none` предотвращает циклическую репликацию
- Использование составного PRIMARY KEY с node позволяет избежать конфликтов
- Каждый узел вставляет данные со своим идентификатором

3. Переключение и каскадирование

Выполнено переключение (failover) и настроена каскадная репликация.

Процесс failover:

- `pg_promote()` переводит реплику в режим мастера
- Бывший мастер пересоздаётся через `pg_basebackup` от нового мастера
- Двунаправленное переключение продемонстрировало гибкость топологии

Каскадная репликация:

- Реплика beta может быть источником для gamma
- `recovery_min_apply_delay = 10s` создаёт отложенную реплику
- При сбое промежуточного узла каскадная реплика автоматически продолжает работу

Выводы

Создание кластеров: Использование `initdb` для создания отдельных кластеров проще и надёжнее, чем копирование существующего. Позволяет полностью контролировать настройки каждого экземпляра.

Роль replication: Создание специальной роли с правами `REPLICATION` обеспечивает безопасное подключение реплик. Использование пароля добавляет уровень защиты.

Параметр -C в pg_basebackup: Автоматически создаёт слот репликации, упрощая настройку. Опция `-R` генерирует конфигурацию standby.

Синхронная vs асинхронная: Синхронная репликация гарантирует целостность, но снижает производительность. Выбор зависит от требований к RTO/RPO.

Логическая репликация: Гибче физической, позволяет селективную репликацию таблиц. Требует PRIMARY KEY и подходит для миграций между версиями.

Каскадирование: Снижает нагрузку на мастер при большом количестве реплик. Отложенные реплики защищают от логических ошибок.

Автоматизация: Для production-систем необходимы инструменты типа Patroni для автоматического failover и управления кластером.